

Mục tiêu

- Nắm được cơ bản xử lý chuỗi ký tự.
- Lưu ý pointer mảng 1 chiều và pointer struct.
- Nắm được các kiến thức cơ bản về đọc ghi file text và con trỏ hàm (function pointer).

Yêu cầu

- Bài A: hoàn thành 1 bài bất kỳ trong các bài 1, 2, 3.
- Bài H: hoàn thành đủ 4 bài (tính thành 10 bài).
- Mỗi bài làm trong 1 project riêng, tất cả các bài làm trong 1 solution.

Bài tập

Bài 1

Làm hoàn thiện bài ví dụ con trỏ hàm với file data.txt là:

```
23120001;Phan Van A
23120017;Nguyen Van C
23120051;Tran Thi B
23120005;Le Thi M
```

Bài 2

Xây dựng struct **Nguoi** để hàm main sau chạy đúng:

```
#define MAXLEN 31
struct Nguoi
{
    char HoTen[MAXLEN];
    char DiaChi[MAXLEN];
};

Nguoi* TaoNguoi(const char *pHt, const char *pDc)
{
}

char* XuatNguoi(Nguoi *pN)
{
}

void ChuanHoaTen(Nguoi *pN)
{
}

void main()
{
    int n = 3;
    Nguoi* pDsNguoi = (Nguoi**)malloc(n * sizeof(Nguoi*));
    pDsNguoi[0] = TaoNguoi(" LE THI duNG ", "HCM");
    pDsNguoi[1] = TaoNguoi(" ngUYen VAN ngUyEn ", "HN");
    pDsNguoi[2] = TaoNguoi(" tRan trung trUc ", "DN");

    printf("Danh sach Nguoi moi tao:\n");
    for (int i=0; i<n; ++i)
    {
        printf("%s\n", XuatNguoi(pDsNguoi[i]));
    }
}
```

```

    }

    printf("\nDanh sach Nguoi sau khi chuan hoa ten:\n");
    for (int i=0; i<n; ++i)
    {
        ChuanHoaTen(pDsNguoi[i]);
        printf("%s\n", XuatNguoi(pDsNguoi[i]));
    }

    for (int i=0; i<n; ++i)
    {
        free(pDsNguoi[i]);
    }
    free(pDsNguoi);
    getch();
}

```

Kết quả:

```

Danh sach Nguoi moi tao:
LE THI duNG      [HCM]
ngUYen VAN ngUYen [HN]
tRan trung trUc   [DN]

Danh sach Nguoi sau khi chuan hoa ten:
Le Thi Dung [HCM]
Nguyen Van Nguyen [HN]
Tran Trung Truc [DN]

```

Bài 3

Xây dựng struct **PhanSo** để hàm main sau chạy đúng:

```

struct PhanSo
{
    int tu, mau;
};

char* XuatPhanSo(PhanSo *pPS)
{
}

PhanSo* TongPhanSoTuChuoi(const char *s)
{
}

void main()
{
    PhanSo *pPS = TongPhanSoTuChuoi("14/56+24/45");
    printf("Tong cua Phan so duoc tu chuoi '14/56+24/45' la:\n");
    printf("\t%s\n", XuatPhanSo(pPS));
    free(pPS);

    getch();
}

```

Kết quả:

```

Tong cua Phan so duoc tu chuoi '14/56+24/45' la:
47/60

```

Bài 4 (tính thành 7 bài)

Viết chương trình thao tác với danh sách các tài khoản người dùng

Thông tin tài khoản:

1. Tên đăng nhập
 - a. Tên đăng nhập ko được chứa chữ “admin”
2. Mật khẩu (khi nhập thì cần nhập mật khẩu 2 lần)
 - a. Từ 6 kí tự trở lên
 - b. Có kí tự in hoa
 - c. Có kí tự a→z, A→Z
 - d. Có số
 - e. Có 1 trong các dấu phẩy, chấm, hỏi, hai chấm, chấm phẩy, ngã, gạch ngang, gạch dưới
3. Họ tên
 - a. Khi nhập, nhập chung 1 chuỗi họ tên, vì là họ tên nên ít nhất phải có 2 từ. Khi xuất, xuất riêng 3 phần: họ, tên lót và tên
4. Email
 - a. Phải chứa dấu @ và dấu .
5. Ngày tháng năm sinh
 - a. Khi nhập thì nhập ngày tháng năm sau, khi xuất thì xuất tuổi

Các chức năng:

1. Nhập vào danh sách tài khoản.
2. Xuất danh sách tài khoản ra màn hình.
3. Nhập vào 1 chuỗi, xuất tất cả các tài khoản mà trong họ tên có chứa chuỗi đó. VD: “phan” thì xuất tài khoản “Phan Van N”.
4. Nhập vào 1 tài khoản mới
 - a. Nếu tên đăng nhập đã có, thì xuất thông tin tài khoản đã có ra màn hình
 - b. Nếu tên đăng nhập chưa có, thì thêm tài khoản vào cuối danh sách
5. Nhập vào 1 tên đăng nhập. Xóa tài khoản có tên đăng nhập đó ra khỏi danh sách
6. Sử dụng con trỏ hàm để sắp xếp danh sách tài khoản cho phép lựa chọn chế độ:
 - a. Tăng dần hay giảm dần
 - b. Theo tên đăng nhập
 - c. Theo tuổi
 - d. Theo họ tên
7. Chúc mừng sinh nhật các tài khoản có ngày sinh trong tháng này (tháng 11).

Nội dung

Thao tác đọc ghi file kiểu văn bản trong C

- ✓ Vấn đề thao tác file theo kiểu văn bản là khá dễ tiếp cận vì nó hầu như giống hệt thao tác với màn hình và thiết bị nhập chuẩn (bàn phím).
- ✓ Đọc

Số lượng phần tử thực sự đọc được.

Con trỏ file đang mở.

```
int fscanf(
    FILE *stream,
    const char *format [,
        argument ]...
);
```

Chuỗi format, định dạng cách đọc.

Các tham số thêm, số lượng tùy ý và được sử dụng dựa vào chuỗi format.

✓ Ghi

Số lượng byte thực sự ghi được.

Con trỏ file đang mở.

```
int fprintf(
    FILE *stream,
    const char *format [,
        argument ]...
);
```

Chuỗi format, định dạng chuỗi sẽ ghi.

Các tham số thêm, số lượng tùy ý và được sử dụng ghép vào chuỗi format.

- ✓ Trong chuỗi format thường sử dụng chuỗi đặc tả chuyển dạng để đổi các kiểu giá trị khác ra chuỗi và ngược lại ("%d" chuyển số int sang chuỗi và ngược lại). Cấu trúc tổng quát của chuỗi đặc tả: %[-][fw][.pp]ký tự chuyển dạng
 - [-]: định dạng việc chuỗi kết quả sẽ dồn về bên phải hay trái trong chiều rộng cho phép (định dạng bởi giá trị fw).
 - [fw]: định dạng bề rộng của chuỗi kết quả.
 - [.pp]: định dạng chỉ áp dụng cho giá trị số thực, quy định số chữ số chính xác phần thập phân.
- ✓ Ngoài ra còn có các hàm làm việc theo từng ký tự: fputc, fgetc.
- ✓ Ví dụ đơn giản

```
void main()
{
    FILE* f = NULL;

    if((f = fopen("test.txt", "w+")) != NULL)
    {
        int n=2345;
```

```

        //ghi vào file tương tự xuất ra màn hình
        fprintf(f, "%d", n);

        //trở về đầu file
        fseek(f, 0, SEEK_SET);

        int m;

        //đọc giá trị vừa ghi
        fscanf(f, "%d", &m);

        //đóng file
        fclose(f);

        printf("%d", m);

        getch();
    }
}

```

Thao tác đọc ghi file kiểu văn bản trong C++

```

int main()
{
    const char *pathFile = "test.txt";
    ifstream fIn;
    ofstream fOut;
    int n = 1234;
    /// mở file để ghi với cờ tạo mới hoàn toàn
    fOut.open(pathFile, ios::trunc);
    fOut << n;
    /// đóng file
    fOut.close();
    /// mở file để ghi với cờ về vị trí đầu
    fIn.open(pathFile, ios::beg);
    int r = -1;
    fIn >> r;
    cout << "read: " << r << endl;
    fIn.close();
}

```

Con trỏ hàm (function pointer)

Trong C++, con trỏ hàm (function pointer) được sử dụng để lưu trữ địa chỉ của một hàm.

Xem ví dụ sau:

```

#include<iostream>
#include<fstream>
#include<cstring>
#include<cstdlib>
#include<cstdio>

using namespace std;

#define MAXLEN 100

struct SinhVien
{
    int MSSV;
    char HoTen[51];
};

```



```

char* SVToString(const SinhVien &sv)
{
    /// xuất thông tin sinh viên ra chuỗi
    /// định dạng: <MSSV> - <HoTen>
}

void PrintArr(SinhVien *arr, const int &n)
{
    for (int i = 0; i < n; i++)
    {
        cout << SVToString(arr[i]) << endl;
    }
}

/// định nghĩa con trỏ hàm
typedef int(*SVComparer)(SinhVien, SinhVien);

/// hàm so sánh sinh viên dựa vào mssv
int SVC1(SinhVien sv1, SinhVien sv2)
{
    if (sv1.MSSV > sv2.MSSV)
    {
        return 1;
    }
    if (sv1.MSSV < sv2.MSSV)
    {
        return -1;
    }
    return 0;
}

/// hàm so sánh sinh viên dựa vào họ tên
int SVC2(SinhVien sv1, SinhVien sv2)
{
    return strcmp(sv1.HoTen, sv2.HoTen);
}

/// hàm sắp xếp sinh viên
void SortAsc(SinhVien *arrSV, const int &n, SVComparer svc)
{
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            if (svc(arrSV[i], arrSV[j]) > 0)
            {
                SinhVien svTemp = arrSV[i];
                arrSV[i] = arrSV[j];
                arrSV[j] = svTemp;
            }
        }
    }
}

/// có thể sử dụng con trỏ hàm không cần định nghĩa trước
void SortDesc(SinhVien *arrSV, const int &n, int (*svSoSanh)(SinhVien, SinhVien))
{
    /// sắp xếp giảm dần
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            if (svSoSanh(arrSV[i], arrSV[j]) < 0)
            {
                SinhVien svTemp = arrSV[i];
                arrSV[i] = arrSV[j];
            }
        }
    }
}

```

```

        arrSV[j] = svTemp;
    }
}

SinhVien* LoadSV(const char *path, int &n)
{
    /// đọc danh sách sinh viên từ file
    /// trả về pointer trỏ đến mảng SinhVien đọc được
    /// n là số sinh viên đọc được
}

int main()
{
    const char *path = "data.txt";
    int n = 0;
    SinhVien *arrSV1 = LoadSV(path, n);
    SinhVien *arrSV2 = LoadSV(path, n);
    cout << "*****Arr begin*****" << endl;
    PrintArr(arrSV1, n);
    SortAsc(arrSV1, n, SVC1);
    cout << "*****Arr sorted mode 1*****" << endl;
    PrintArr(arrSV1, n);
    SortDesc(arrSV2, n, SVC2);
    cout << "*****Arr sorted mode 2*****" << endl;
    PrintArr(arrSV2, n);
    system("pause");
}

```

Hướng dẫn cơ bản Bài 4

Tạo 5 file, viết các lệnh #ifndef và lệnh #include tương ứng

1. TaiKhoan.h
2. TaiKhoan.cpp
3. MangTaiKhoan.h
4. MangTaiKhoan.cpp
5. Main.cpp

Khai báo struct Ngay, TaiKhoan

```

// 3 dòng lệnh này để tránh include 2 lần
#ifndef _TAIKHOAN_H_
#define _TAIKHOAN_H_

// Chuyển qua hệ thống nhập xuất C++
#include<iostream>
using namespace std;
// Ko #include<stdio.h> nữa

// Có tên đăng nhập, mật khẩu, email, họ tên, ngày tháng năm sinh
// => struct Ngay
struct Ngay
{
    int ngay, thang, nam;
};

```

```
// => struct TaiKhoan
struct TaiKhoan
{
    char tenDangNhap[31];    // chuỗi tối đa 31-1=30 kí tự
    char matKhai[31];        // chuỗi tối đa 31-1=30 kí tự
    char hoTen[51];          // chuỗi tối đa 51-1=50 kí tự
    char email[51];          // chuỗi tối đa 51-1=50 kí tự
    Ngay ntng;               // ngày tháng năm sinh
};
```

Hàm nhập thông tin tài khoản

```
// Tên hàm: nhập tài khoản
// Tham số: tài khoản chứa giá trị nhập vào
// Trả về: ko trả về gì hết
void Nhap(TaiKhoan &tk);
```

```
// Tên hàm: kiểm tra tên đăng nhập
// Tham số: tên đăng nhập
// Trả về: 0: ko hợp lệ; 1: hợp lệ -> int
int KiemTraTenDangNhap(char tenDangNhap[]);

// Tên hàm: kiểm tra mật khẩu
// Tham số: mật khẩu
// Trả về: 0: ko hợp lệ; 1: hợp lệ -> int
int KiemTraMatKhai(char matKhai[]);

// Tên hàm: kiểm tra họ tên
// Tham số: họ tên
// Trả về: 0: ko hợp lệ; 1: hợp lệ -> int
int KiemTraHoTen(char hoTen[]);

// Tên hàm: kiểm tra email
// Tham số: email
// Trả về: 0: ko hợp lệ; 1: hợp lệ -> int
int KiemTraEmail(char email[]);

// Tên hàm: nhập ngày tháng năm
// Tham số: ngày tháng năm chứa giá trị nhập vào
// Trả về: ko có
void Nhap(Ngay &ng);
```



```

void Nhap(TaiKhoan &tk)
{
    do
    {
        //printf("Nhap ten dang nhap");
        cout << "Nhap ten dang nhap: ";
        //gets(tk.tenDangNhap);
        cin.get(tk.tenDangNhap, 30, '\n');
    } while (KiemTraTenDangNhap(tk.tenDangNhap) == 0);

    do
    {
        // printf("Nhap mat khau: ");
        cout << "Nhap mat khau: ";
        // gets(tk.matKhau);
        cin.get(tk.matKhau, 30, '\n');
    } while (KiemTraMatKhau(tk.matKhau) == 0);

    do
    {
        cout << "Nhap ho ten: ";
        cin.get(tk.hoTen, 50, '\n');
    } while (KiemTraHoTen(tk.hoTen) == 0);

    do
    {
        cout << "Nhap email: ";
        cin.get(tk.email, 50, '\n');
    } while (KiemTraEmail(tk.email) == 0);

    cout << "Nhap ngay thang nam sinh";
    Nhap(tk.ntns);
}

```

```

int KiemTraTenDangNhap(char tenDangNhap[])
{
    // B1: Copy qua chuỗi s, tránh thay đổi chuỗi tenDangNhap
    char s[31];
    strcpy(tenDangNhap, s);
    // B2: Chuyển s thành chữ thường
    strlwr(s);
    // B3: Dùng hàm strstr kiểm tra chuỗi có nằm trong chuỗi
    char *p = strstr(s, "admin");
    if (p == NULL) // có
        return 0;
    return 1;
}

```

```
int KiemTraMatKhai(char matKhai[])
{
    // B1: Mật khẩu >=6 kí tự
    int doDai = strlen(matKhai);
    if (doDai < 6)
        return 0;

    // B2: Mật khẩu có kí tự in hoa
    // Duyệt qua từng kí tự [i] trong chuỗi,
    // xét xem kí tự đó có nằm trong đoạn A-Z ko
    int coKiTuHoa = 0; // giả sử chưa có kí tự in hoa
    for (int i = 0; i < doDai; i++)
    {
        if (matKhai[i] >= 'A' && matKhai[i] <= 'Z')
            coKiTuHoa = 1;
    }
    if (coKiTuHoa == 0)
        return 0;
```

```
    // B3: Mật khẩu có kí tự chữ cái
    int coKiTu = 0;
    for (int i = 0; i < doDai; i++)
    {
        if (matKhai[i] >= 'a' && matKhai[i] <= 'z')
            coKiTu = 1;
        if (matKhai[i] >= 'A' && matKhai[i] <= 'Z')
            coKiTu = 1;
    }
    if (coKiTu == 0)
        return 0;

    // B4: Mật khẩu có kí tự số
    int coSo = 0;
    for (int i = 0; i < doDai; i++)
    {
        if (matKhai[i] >= '0' && matKhai[i] <= '9')
            coKiTu = 1;
    }
    if (coSo == 0)
        return 0;
```

```

// B5: Mật khẩu có các dấu yêu cầu
int coDau = 0;
for (int i = 0; i < doDai; i++)
{
    if (matKhau[i] == ',' || matKhau[i] == '.' || matKhau[i] == ':')
        coKiTu = 1;
}
if (coSo == 0)
    return 0;
// B6: Lọt đến đây là thỏa hết các yêu cầu
return 1;
}

```

```

int KiemTraHoTen(char hoTen[])
{
    // B1: họ tên có 2 từ -> có khoảng trắng
    // Duyệt qua từng ký tự [i] trong chuỗi
    int coKhoangTrang = 0; // giả sử chưa có khoảng trắng
    int doDai = strlen(hoTen);
    for (int i = 0; i < doDai; i++)
    {
        if (hoTen[i] == ' ')
            coKhoangTrang = 1;
    }
    if (coKhoangTrang == 0)
        return 0;
    return 1;
}

```

```

int KiemTraEmail(char email[])
{
    // B1: Kiểm tra có @ ko, nếu ko return 0
    // B2: Kiểm tra có . ko, nếu ko return 0
    return 1;
}

```

Hàm xuất thông tin 1 tài khoản

```

// Tên hàm: xuất tài khoản
// Tham số: tài khoản cần xuất thông tin
// Trả về: ko trả về gì hết
void Xuat(TaiKhoan tk);

```

```
// Tên hàm: xuất ngày tháng năm
// Tham số: ngày tháng năm cần xuất
// Trả về: ko có
void Xuat(Ngay ng);
```

```
void Xuat(TaiKhoan tk)
{
    // printf("Ten dang nhap: %s\n", tk.tenDangNhap);
    cout<<"Ten dang nhap: "<< tk.tenDangNhap<<"\n";
    // printf("Mat khau: %s\n", tk.matKhau);
    cout<<"Mat khau: "<<tk.matKhau << "\n";
    // printf("Ho ten: %s\n", tk.hoTen);
    cout<<"Ho ten: "<<tk.hoTen << "\n";
    // SV tự làm phần tách họ, tên lót và tên ra 3 phần riêng
    // printf("Email: %s\n", tk.email);
    cout<<"Email: "<<tk.email << "\n";
    // printf("NTNS: ");
    cout<<"NTNS: ";
    Xuat(tk.ntns);
    // printf("\nTuoi: %d\n", 2015 - tk.ntns.nam);
    cout<<"\nTuoi: "<<2015 - tk.ntns.nam << "\n";
}
```

```
void Xuat(Ngay ng)
{
    // printf("%d/%d/%d", ng.ngay, ng.thang, ng.nam);
    cout << ng.ngay << "/" << ng.thang << "/" << ng.nam;
}
```

Bây giờ, chúng ta sẽ viết các hàm trong 2 file MangTaiKhoan.h và MangTaiKhoan.cpp

```
#ifndef _MANGTK_H_
#define _MANGTK_H_

#include "TaiKhoan.h"

// Tên hàm: Nhập mảng tài khoản
// Tham số: mảng, số phần tử
// Trả về: ko có
void Nhap(TaiKhoan *a, int &n);

// Tên hàm: xuất mảng tài khoản
// Tham số: mảng, số phần tử
// Trả về: ko có
void Xuat(TaiKhoan *a, int n);
```

```
// Tên hàm: liệt kê tài khoản theo tên
// Tham số: mảng, số phần tử, chuỗi tìm kiếm
// Trả về: ko có
void LietKeTheoTen(TaiKhoan *a, int n, char ten[]);

// Tên hàm: thêm tài khoản
// Tham số: mảng, số phần tử, tài khoản mới cần thêm
// Trả về: nếu tài khoản đã có: trả về vị trí;
// nếu tài khoản chưa có: trả về -1
int ThemTaiKhoan(TaiKhoan *a, int &n, TaiKhoan tk);

// Tên hàm: xóa tài khoản
// Tham số: mảng, số phần tử, tên đăng nhập
// Trả về: ko có
void XoaTaiKhoan(TaiKhoan *a, int &n, char tenDangNhap[]);
#endif
```



```
void Nhap(TaiKhoan *&a, int &n)
{
    // B1: Nhập số lượng tài khoản
    cout << "Nhap n: ";
    cin >> n;
    // B2: Cấp vùng nhớ cho mảng a
    a = new TaiKhoan[n];
    // B3: Kiểm tra cấp vùng nhớ có thành công ko
    if (a == NULL)
        return;
    // B4: Duyệt từ 0 đến n-1, gọi hàm nhập 1 tài khoản
    for (int i = 0; i < n; i++)
    {
        cout << "Nhap tai khoan thu " << i << "\n";
        Nhap(a[i]);
    }
}
```

```
void Xuat(TaiKhoan *a, int n)
{
    // B1: Kiểm tra n=0, báo ko có tài khoản nào
    if (n == 0)
    {
        cout << "Ko co tai khoan nao trong danh sach\n";
        return;
    }
    // B2: Duyệt từ đầu đến cuối mảng, gọi hàm xuất tài khoản
    for (int i = 0; i < n; i++)
    {
        cout << "Tai khoan thu " << i << "\n";
        Xuat(a[i]);
        cout << "\n";
    }
}
```

```
void LietKeTheoTen(TaiKhoan *a, int n, char ten[])
{
    // B1: Kiểm tra n=0, báo ko có tài khoản nào
    // B2: Duyệt từ đầu đến cuối mảng,
    for (int i = 0; i < n; i++)
    {
        char s[51];
        // Copy sang s để tránh thay đổi họ tên của a[i]
        strcpy(a[i].hoTen, s);
        // Chuyển s thành chữ thường
        // Kiểm tra ten có nằm trong s
        char *p = strstr(s, ten);
        if (p != NULL)
        {
            // Tìm thấy, gọi hàm xuất tài khoản a[i]
        }
    }
}
```